Лабораторная №10 — Блокирующий сервер и

Цель

Изучить поведение планировщика ОС при блокирующих вызовах `accept()`/`recv()`, исследовать состояние `TIME_WAIT`, освоить флаг `SO_REUSEADDR`.

Код

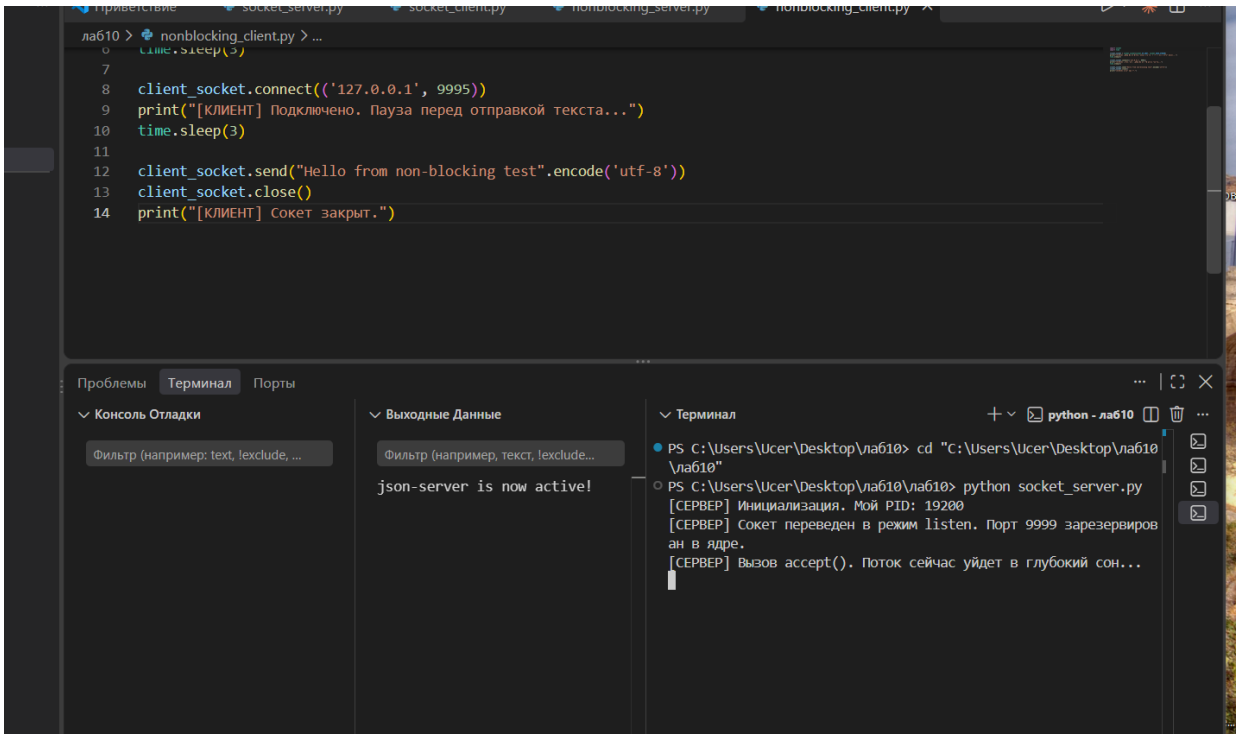
`socket_server.py`

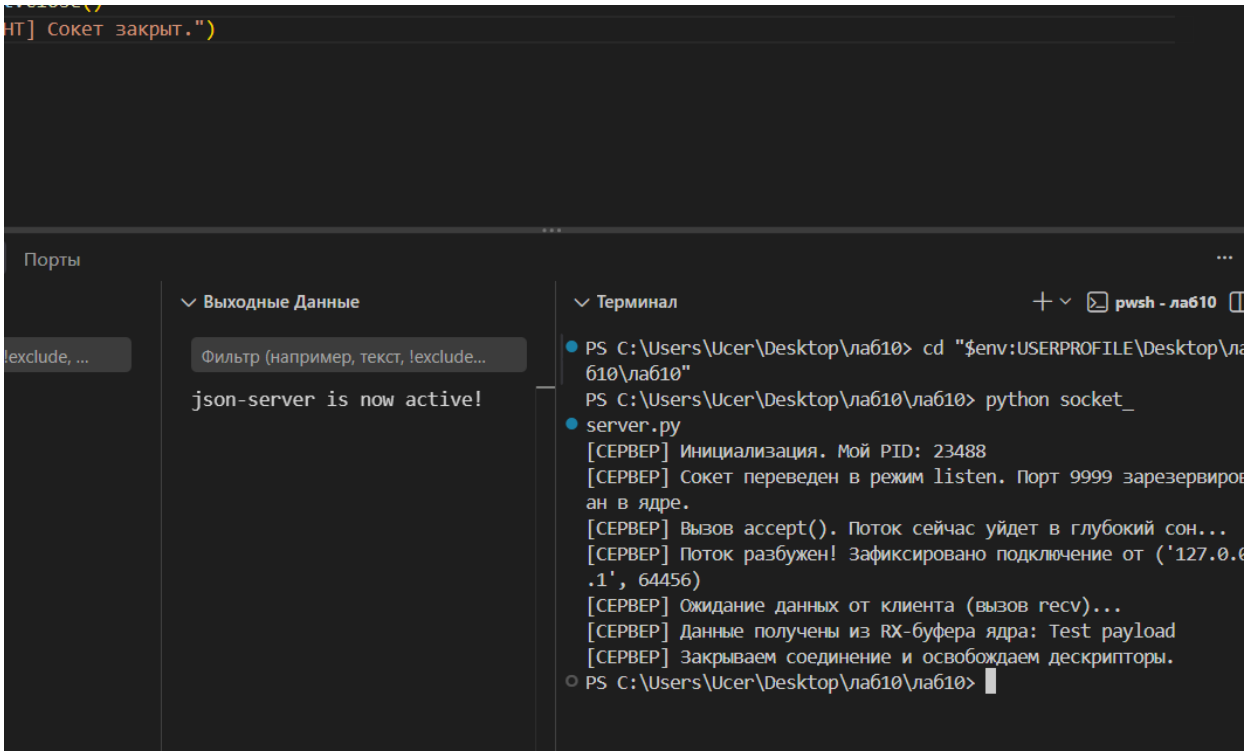
```python
import socket, os
PORT = 9999
s = socket.socket()
# s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)  # закомментировано для эксперимента
print(f"[СЕРВЕР] PID: {os.getpid()}")
s.bind(('127.0.0.1', PORT)); s.listen(1)
print("[СЕРВЕР] accept() — уходим в сон...")
c, a = s.accept()
print(f"[СЕРВЕР] Проснулся! Клиент: {a}")
d = c.recv(1024)
print(f"[СЕРВЕР] Данные: {d.decode()}")
c.close(); s.close()
```

`socket_client.py`

```python
import socket, time
c = socket.socket()
time.sleep(3)
c.connect(('127.0.0.1', 9999))
time.sleep(3)
c.send("Test payload".encode())
c.close()
```

Результаты





Контрольные вопросы

1. Сон на Accept() и 0% CPU

Accept() — это блокирующий системный вызов. Когда соединений нет, ядро переводит поток из состояния Running в состояние Waiting (Sleep). Поток убирается из очереди планировщика, ему не выдаётся квант времени — поэтому CPU не тратится. Просыпается он только когда придёт новый пакет (событие от сети), тогда ядро снова ставит его в очередь на выполнение.

2. Почему порт занят после закрытия сервера
Сокет после закрытия уходит в состояние TIME_WAIT (на стороне сервера это видно как LISTEN с флагом в netstat/ss). Ядро держит запись в таблице портов ещё некоторое время, чтобы не потерять запоздавшие пакеты от старых соединений. Поэтому новый bind() получает ошибку «Address already in use».

3. Что делает SO_REUSEADDR
Флаг разрешает повторно привязаться к порту, который находится в TIME_WAIT. На уровне таблиц портов ядро вместо отказа выдаёт новый сокет и переиспользует запись. Быстрый перезапуск сервера после этого работает без ошибки.

Лабораторная №11 — Неблокирующий сервер

Цель

Изучить неблокирующий режим сокета, `BlockingIOError` (`EAGAIN`), накладные расходы polling.

Код

`nonblocking_server.py`

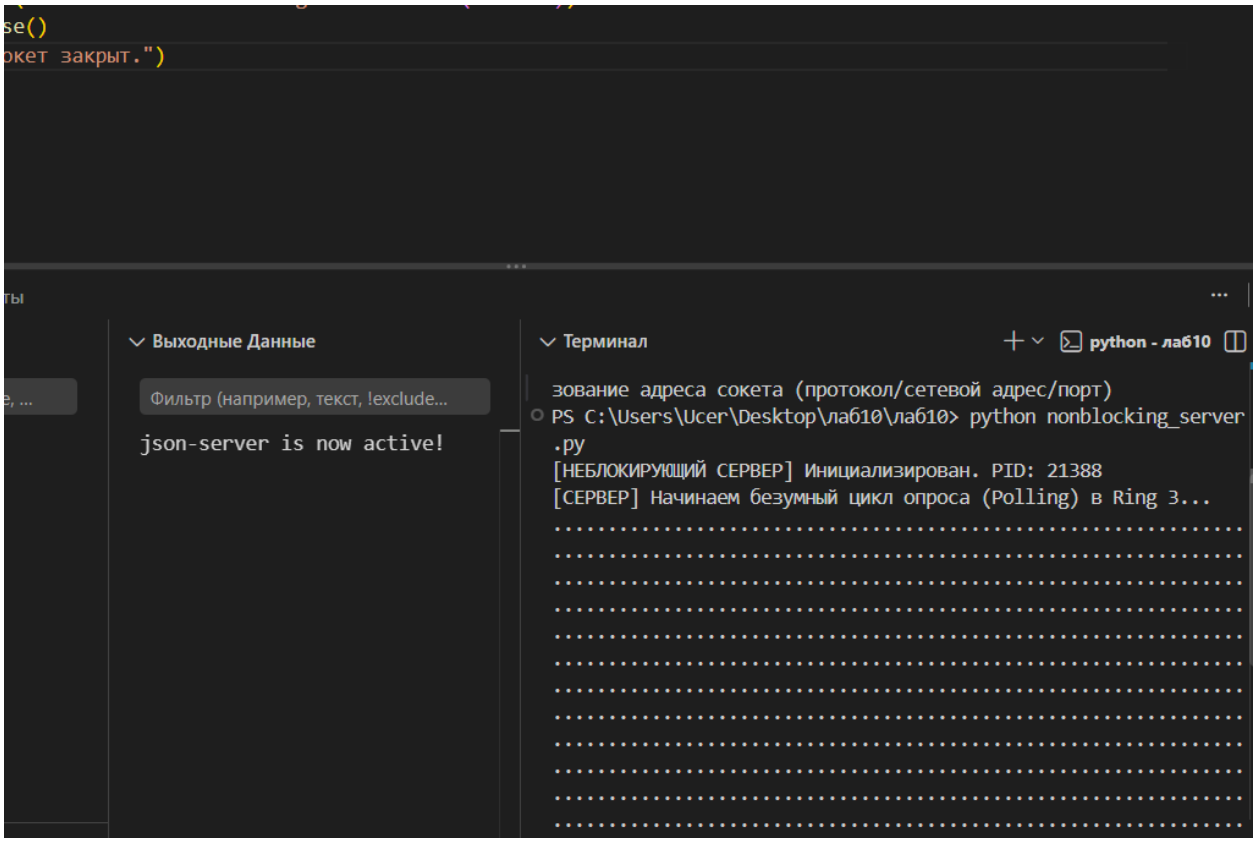
```
python
import socket, os, time
PORT = 9995
s = socket.socket()
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(('127.0.0.1', PORT)); s.listen(1)
s.setblocking(False)
print(f"[НБ-СЕРВЕР] PID: {os.getpid()}")
while True:
    try:
        c, a = s.accept()
        print(f"\n[НБ-СЕРВЕР] КЛИЕНТ: {a}")
        break
    except BlockingIOError:
        print(".", end="", flush=True)
        time.sleep(0.01)   # <- убираем для 100% CPU
```

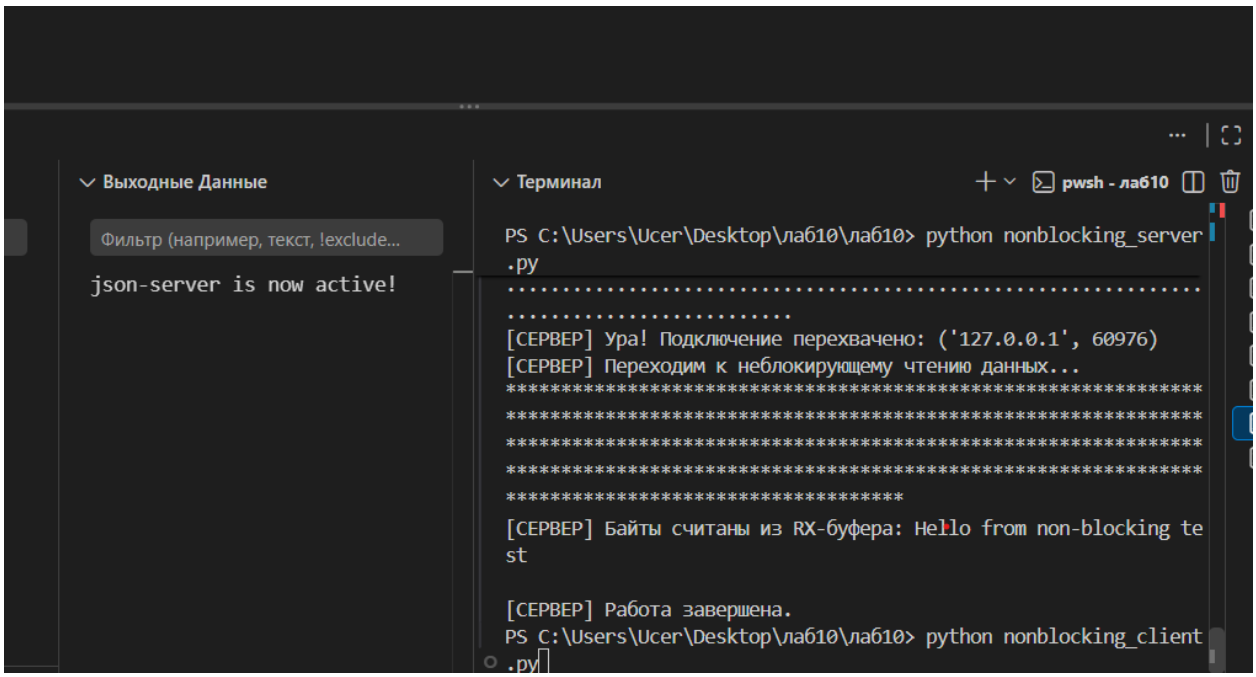
```
c.setblocking(False)
while True:
    try:
        d = c.recv(1024)
        if d: print(f"\n[НБ-СЕРВЕР] ДАННЫЕ: {d.decode()}"); break
    except BlockingIOError:
        print("*", end="", flush=True)
        time.sleep(0.01)
c.close(); s.close()
```

nonblocking_client.py

```
python
import socket, time
time.sleep(3)
c = socket.socket()
c.connect(('127.0.0.1', 9995))
time.sleep(3)
c.send("Hello from non-blocking test".encode())
c.close()
```

Результаты





Контрольные вопросы .

EWOULDBLOCK / EAGAIN
Ядро говорит: «данных нет, приходи позже». Программа сама попросила неблокирующий режим — значит, ждать её никто не обязан. Возвращаем ошибку, а не паузу.

2. Почему 100% CPU без sleep
Пустой цикл: accept() → ошибка → accept() → ошибка… Каждый вызов — переход Ring 3 → Ring 0 и обратно (сохранение регистров, проверки). Полезной работы ноль, но миллионы переходов в секунду = 100% загрузки. sleep(0.01) даёт ядру паузу.

3. Как делают nginx / Node.js
Не крутят пустой цикл, а используют мультиплексирование: один вызов — «следи за всеми сокетами, разбуди при событии». Linux — **epoll**, Windows — **IOCP**, BSD/macOS — **kqueue**. Поток спит в ядре, просыпается только на реальное событие. Один поток — тысячи соединений, без 100% CPU.